Secure Data Exchange Protocol for IPFS

1st Csaba Nagy
Faculty of Informatics
University of Debrecen
Debrecen, Hungary
nagy.csaba@inf.unideb.hu

2nd Andrea Huszti
Faculty of Informatics
University of Debrecen
Debrecen, Hungary
huszti.andrea@inf.unideb.hu

3rd Norbert Oláh
Faculty of Informatics
University of Debrecen
Debrecen, Hungary
olah.norbert@inf.unideb.hu

***Abstract*—The adoption of decentralized Web3 architectures has led to the growing use of IPFS for distributed storage. IPFS nevertheless raises privacy concerns, as network participants may monitor content requests and associate them with specific users or data objects. While Triple Hashing has been proposed to obscure Content Identifiers (CIDs), additional protection mechanisms are still required against profiling and active network-level attacks. This paper proposes an enhanced privacy framework for IPFS that integrates digital signatures and asymmetric encryption to secure the Triple Hash exchange. Furthermore, we introduce decoy requests to mitigate user profiling and a novel encrypted caching mechanism that allows resource-rich nodes to serve content without compromising data privacy. Formal verification using ProVerif was used to evaluate the security properties of the proposed protocol, including content authenticity and key secrecy.**

***Index Terms*—IPFS, Triple Hashing, Peer-to-Peer Privacy, Man-in-the-Middle, Decoy Requests, Encrypted Caching.**

I. Introduction

In recent decades, the Internet has undergone an explosive expansion. The first stage of the World Wide Web (WWW) was Web 1.0. During this era, only a few individuals created content. Individual web pages contain predominantly static content, and these websites are typically hosted on web servers managed by Internet Service Providers (ISPs) or through free web hosting services. Web 2.0 was the second stage, a collective term for internet services. They are typically community-based, and a significant change is that the users create content or share information. In contrast to Web 1.0, where the service provider provides the static content, in Web 2.0 services, the server host only provides the framework. Thus, the users upload, create, share, or review the content. Users typically communicate and build relationships with each other. Today, Web3 attempts to establish a decentralized platform capable of rivalling cloud infrastructures that underpin the current internet landscape. Nonetheless, Web3's current scope is bounded to applications facilitating content creation and sharing, decentralized finance, and decentralized communication. However, the present Internet is based on primary cloud providers such as Google, Amazon, and Microsoft. These providers provide a wide range of web services to many users globally. However, this centralized cloud infrastructure is vulnerable to targeted attacks [1]. Furthermore, these providers set the terms of service with little to no competition. Many propositions and techniques exist to achieve Web 3.0 and decentralize our networks, such as blockchain [2] or the InterPlanetary File System (IPFS) [3]. IPFS is a distributed Peer-to-peer (P2P) file system where multiple peers need to work together to find the content. However, these systems meet an obvious privacy issue: a malicious peer can monitor the request that others are searching for. Moreover, the malicious peer can create multiple virtual nodes to enhance its ability to eavesdrop on lookup requests.

II. Related Work

The InterPlanetary File System was introduced by Benet [3] as a content-addressed, peer-to-peer distribution file system that combines ideas from Distributed Hash Tables (DHT), BitTorrent swarm, and Git Repository. Daniel and Tschorsch [4] provide a comprehensive survey of IPFS and related next-generation P2P data networks, such as Ethereum Swarm and GNUnet, noting that IPFS lacks dedicated privacy mechanisms. Content identifiers are publicly visible in DHT lookups, and any participating node can observe which objects are being requested.

Trautwein et al. [5] conducted a large-scale measurement of the IPFS network spanning over 2,700 Autonomous Systems and 152 countries, demonstrating that the Kademlia DHT sustains efficient content routing even at this scale, yet with notable variance in lookup latency and content availability across regions. Henningsen et al. [6] systematically crawled the IPFS Kademlia DHT, mapping its topology and revealing how routing table structures can expose participation patterns. Building on these findings, Balduf et al. [7] demonstrated that monitoring DHT request traffic enables an adversary to learn which content specific peers are accessing, confirming that IPFS "is not a privacy focused network". Beyond passive monitoring, Cholez and Ignat [8] showed that even a low-resource Sybil attack launched from a single machine can deny content access in IPFS by polluting the DHT with forged provider records.

In the broader context of P2P privacy, Tor [9] provides strong anonymity through circuit-based onion routing, where traffic is relayed through multiple encrypted hops so that no single node observes both the sender and the content. While effective for general-purpose anonymous communication, Tor's multi-hop relay design introduces significant latency overhead that is undesirable for bulk file retrieval. Furthermore, while Tor masks the network-level identity, it does not address the

inherent information leakage of content-addressed lookups at the application level, where the requested content identifiers remain exposed to potentially malicious relays.

This limitation necessitates architectural changes within the P2P protocol itself, such as the triple hashing concept introduced by Katsantas et al. [10]. They proposed deriving three cascaded hash-based content identifiers to separate the public search key from the content address. Their protocol modifies the Bitswap [11] exchange so that the owner proves knowledge of CID^2 before the requester reveals CID^1. However, as demonstrated by our analysis in Section III, this protocol lacks mutual authentication between participants, making it vulnerable to Man-in-the-Middle (MITM) attacks in open environments, where an adversary who intercepts the challenge-response messages can replay them and ultimately obtain the object. Our work improves the triple hashing framework with cryptographic primitives to ensure the requester's privacy and robust content authentication. In addition, we introduce decoy requests to defend against profiling attackers, who apply rainbow table attacks. Finally, we propose an encrypted caching protocol for nodes that serve as central elements of the IPFS network and have strong connectivity to numerous other nodes.

III. Enhancing Privacy

Triple Hashing

To enhance privacy in IPFS, we build on the concept of triple hashing [10]. In this approach, the owner of an object (OBJ) creates three Content Identifiers (CIDs) for the OBJ. CID^1 is similar to a standard IPFS CID, containing some metadata and the hash of the OBJ, $H(OBJ)$. In CID^2, the node uses the double hash $H(H(OBJ))$, and in CID^3, the triple hash $H(H(H(OBJ)))$. When the node wants to publish the OBJ, it sends CID^1 to the k closest nodes using the Kademlia DHT [12]. CID^2 is kept secret, while CID^3 is published just like CID^1.

When a node wants to retrieve OBJ from IPFS, it must first compute CID^3, assuming it already has CID^1. The node then starts searching for the node that holds CID^3 using the Kademlia protocol. Once found, the retrieval process begins using Bitswap [11]. As illustrated in Figure 1, in the triple hashing method, the retrieving node (A) sends a WANT_HAVE message with CID^3 to the owner node (B). B responds with a HAVE message, but since CID^3 is not a CID for an OBJ, B includes CID^2 as a challenge and a proof that it knows CID^1. Node A then sends a WANT_BLOCK message with a hashed version of randomized CID^1, preventing the leakage of CID^1. B recognizes this as proof that A knows CID^1, allowing them to begin the object transfer.

This method protects Node A's privacy against curious nodes contacted during the initial search for Node B, as they only know CID^3. Even if they contact Node B, they will only receive CID^2, and there is no way for them to calculate CID^1.

One problem with the triple hash protocol, as shown in Figure 1, is that it does not address MITM attacks. If an attacker replays the messages between Nodes A and B, they could ultimately receive the OBJ from B. This vulnerability exists because the suggested protocol lacks authentication to verify the origin of the packets.

The triple hash protocol builds on the practical implementation of Bitswap. Although [10] did not explicitly mention it, we assume it requires a secure communication channel for each step of the protocol. Without secure communication, the protocol would be vulnerable to eavesdroppers, as shown in Figure 1. If an eavesdropper could intercept the transfer of the OBJ from B to A, the privacy of Node A would be compromised, making all previous privacy efforts useless.

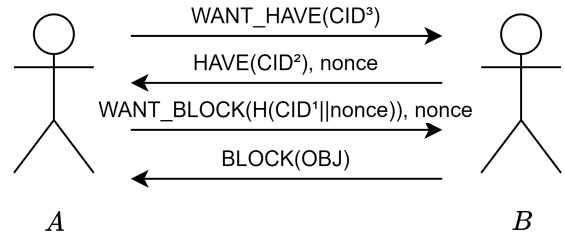

Fig. 1. The triple hash protocol.

Our Solution

The goal of our protocol is to address the MITM attack and provide a solution that does not require a pre-established secure communication channel between every node.

The owner B of the object verifies both CID^1 and CID^3 in the same manner, while the object requester A initiates the search for B using CID^3. Once B is identified, A sends a WANT_HAVE(CID^3) message, as illustrated in Figure 2.B responds with a digital signature over CID^2, PKB, and G^b. Here, SSK denotes the signing key of participant B, PKB is the corresponding public key used for asymmetric encryption, and G^b represents the Diffie–Hellman public value used for deriving the shared session key. It is enough to send only the signature, PKB and G^b, as an honest requester will already know CID^2. After verifying the signature, A responds with a WANT_BLOCK message containing an asymmetric encryption of CID^1, the hash of the session key $H(K)$, and G^a, the Diffie–Hellman public value generated by A. The message is encrypted using PKB. This process allows A to verify that the communicating node possesses CID^2, is capable of producing a valid signature, and can decrypt the exchanged data, reducing the risk of exposing CID^1. Finally, B decrypts the ciphertext and verifies the received CID^1 together with the session key hash $H(K) = H(G^{ab})$. After successful verification, the object is encrypted using K and transmitted to A.

Decoy Requests

While triple hashing is an effective solution to prevent nodes from collecting data on others, it can be compromised if malicious nodes have the resources to build a dictionary that links CID^3 with CID^1. To counteract this profiling risk,

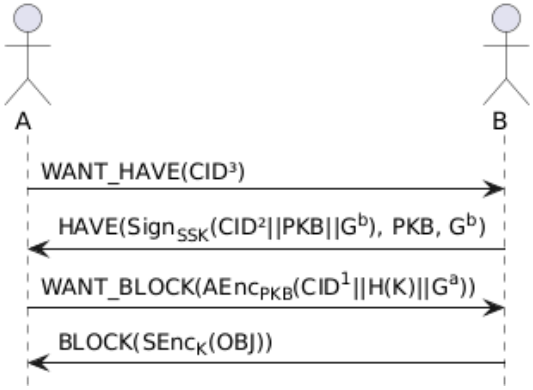

Fig. 2. Our protocol for enhanced privacy.

one approach is to initiate decoy object requests. This method introduces false information into the profiling process.

To support this feature, our protocol is modified in its third and fourth steps, as illustrated in Figure 3. Node A randomly selects an existing CID^1 from a database, calculates the CID^3 and locates a node that knows the corresponding object, following the same process as before. However, in step 3, instead of attaching the Diffie–Hellman parameter G^a and the hash of the session key, A includes a predetermined decoy flag. In response, node B generates random bits and sends them to A, simulating a normal file transfer. Since the object is encrypted during a file transfer, curious nodes cannot distinguish between random bits and actual ciphertext. Finally, node A can immediately discard the decoy message.

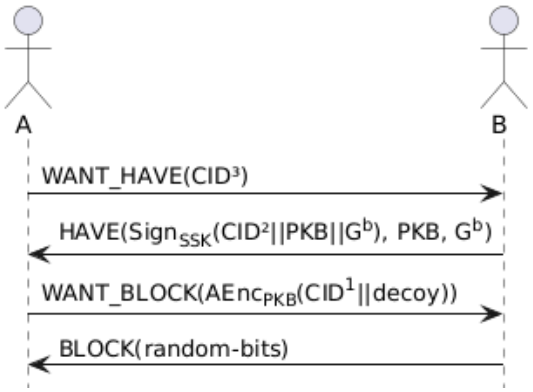

Fig. 3. Our protocol for decoy requests.

Encrypted Caching

To accelerate object retrieval in IPFS, nodes with significant resources can cache frequently requested objects. However, this capability would compromise our privacy requirements if any node could request to cache an object based on its triple-hash CID. To address this, we allow cache nodes to receive, store, and serve objects in an encrypted form.

Figure 4 illustrates how a cache node C requests an object from the owner node B and subsequently serves it to a requester node A. When C attempts to cache an object identified by CID^3, it initiates the protocol with B similarly to a standard retrieval operation. Since C does not know CID^1, the retrieval procedure cannot be completed directly. Instead, C responds with the previously exchanged parameters, extended with an additional Diffie–Hellman public value $G^{c'}$ and the public key PKC. The message is encrypted using B's public key. In return, B sends back a cipher containing the encrypted object and a signature of CID^2 concatenated with C's public key and DH parameter. This signature acts as authorization for C to function as a cache node. The cipher is created using K. The encrypted object itself is generated using the hash in CID^1 as the key and includes both the object and a timestamp concatenated together. The timestamp allows B to set an expiration date for the object and, by regularly updating the timestamp, to alter the appearance of the cipher. Since the final cipher is created using the hash in CID^1 as the key, nodes wishing to retrieve the object will be able to decrypt it, as they already know this key.

After this exchange, C can act as a caching node, serving the encrypted object upon receiving a request for CID^3. This process is also illustrated in Figure 4. C's response to the request will include the authorization received from B, along with B's and C's public keys to make the signature verifiable. If everything is in order, A responds with G^a and $H(K')$, encrypted with asymmetric encryption using C's public key. Using K, node C can encrypt the cached object and send it to A. Finally, since the hash of the object is embedded in CID^1, A can verify the integrity of the received object.

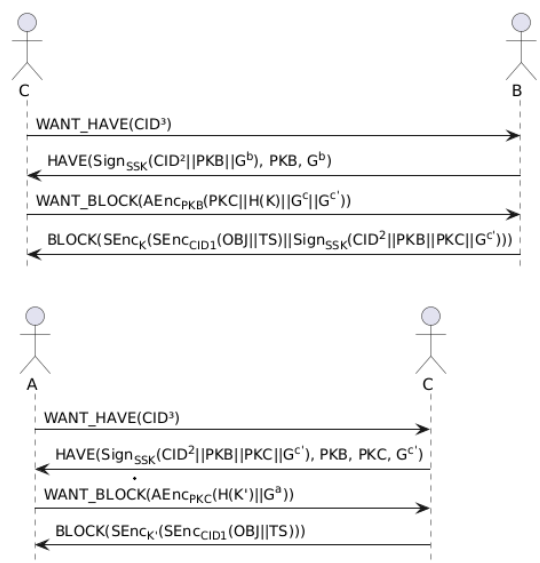

Fig. 4. Our protocol for caching encrypted objects.

IV. Security analysis

In this section, we provide the security analysis of our protocols for IPFS. Firstly, we need to define all security requirements related to the protocol. Some of these requirements can be analyzed using the ProVerif software. This tool is

based on the applied pi-calculus, which was created by Bruno Blanchet, and automatically verifies cryptographic protocols within a formal model [13]. It employs a representation of the protocol based on Horn clauses.

Security requirements

Our primary motivation was that [10] does not explicitly consider Man-in-the-Middle attack scenarios, which may still allow user profiling under certain network conditions. In order to correct and improve this solution, we have collected the security requirements. Our proposition partially covers the requirements of traditional key exchange protocols, although the content-oriented nature of IPFS introduces several important differences.

- Content-based authentication: In our protocol, the identities of the sender and recipient are irrelevant, however, we verify that they have the appropriate resources.
- Key secrecy: An attacker cannot calculate the session key which is the encryption key for the content.
- Key freshness: The protocol runs with a new random session key each time it cannot be successfully exploited using an old, previously used key.
- Forward secrecy: If a session key is compromised, it does not affect the secrecy of previous requests.
- Privacy: When the protocol is running, the requester's privacy is protected against profiling. Furthermore, the content and the requester is unlinkable.

Adversarial model

In the case of our protocol, the adversary's goal is to access the requested content and be able to profile users. We use the Dolev-Yao model [14] for our analysis. In the Dolev-Yao model, an adversary has the following properties:

1) The entire network is assumed to be under adversarial control. The attacker may impersonate legitimate IPFS nodes, intercept and manipulate messages, and generate arbitrary protocol traffic, while remaining limited only by the underlying cryptographic assumptions.
2) The adversary may initiate content retrieval requests with arbitrary nodes and participate in the protocol as a seemingly legitimate peer.

Formal model

We formalize the enhanced privacy and the caching protocols as follows. In the enhanced privacy protocol, we differentiate three processes. In the main process identification numbers for the participants, timestamp, secret and public keys are generated. There are two sub-processes representing the protocols for the Requesting node and content provider. An unbounded number of subprocesses running in parallel are considered. We model the interactions between the requesting node and the content provider. In the caching protocol, we have also formalized the caching node. This results in four subprocesses. In this protocol version, the caching node receives a certificate stating that it is capable of serving the object within the specified time, but cannot reveal the content.

The proposed IPFS protocol was formally analyzed using ProVerif. The corresponding protocol specification and process definitions are publicly available in [15].

Security properties

ProVerif can analyze properties such as reachability, correspondence assertions, and observational equivalence. It is important to note that although ProVerif is sound, it is not complete. Consequently, if ProVerif determines that a property is satisfied, the model guarantees its validity. However, there may still be properties that hold but cannot be proven by ProVerif.

ProVerif uses queries that might be a fact or correspondence for security evaluations. Primarily, we query whether a term sec is secret for the attacker: query attacker (sec). The query is a fact in the case of *reachability*, and we test whether the fact holds.

A *correspondence* has the form $X \rightarrow Y$, meaning that if X holds, then Y must also hold. In the model, events represent important protocol stages, allowing us to verify whether the execution of event a implies that event b has previously occurred. The query $query\ event:\ a(s,t) \rightarrow event:\ b(t,z).$ states that for all s, t, and for every occurrence of $a(s,t)$, there exists a prior occurrence of $b(t,z)$ for some z. To prove a one-to-one relationship, injective correspondences can be used. The query $query\ inj\text{-}event:\ a(s,t) \rightarrow inj\text{-}event:\ b(t,z).$ states that for each occurrence of the event $a(s,t)$, there exists a *distinct* earlier occurrence of the event $b(t,z)$ for some z.

We use the security queries and define events demonstrated by Figure 5.

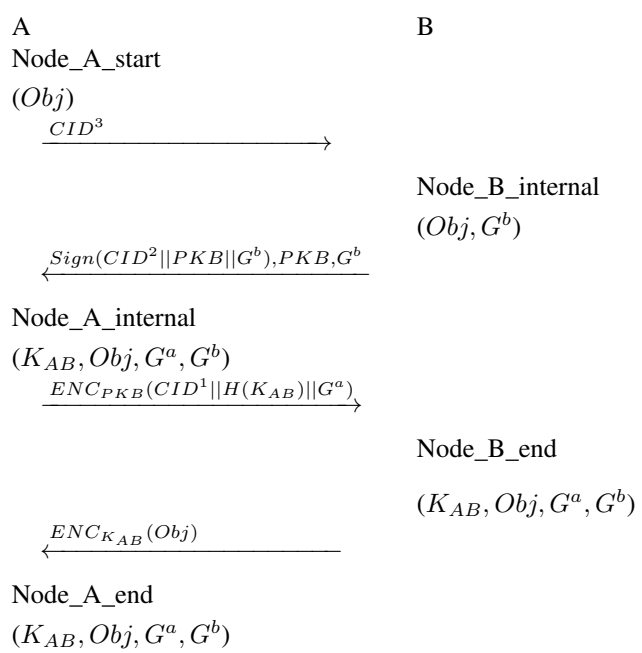

Fig. 5. ProVerif - Enhanced Privacy Protocol Events

We utilize correspondence queries to verify content-based authentication between the requester and the provider. In the model, the event $Node_A_end(k, obj, ga, gb)$ denotes that

node A successfully executes a protocol run with session key k, object obj and Diffie–Hellman parameters ga, gb. The verified correspondence property shows that when A completes a protocol run, B must also have completed the same execution. It also implies that A created the corresponding session state, B processed the same object, and A had previously initiated the protocol for that object. The following verified events represent this property:
$event(Node_A_end(k, obj, ga, gb)) ==> event(Node_B_end(k, obj, ga, gb))$,
$event(Node_B_end(k, obj, ga, gb)) ==> event(Node_A_internal(obj, k, ga, gb))$,
$event(Node_A_internal(obj, k, ga, gb)) ==> event(Node_B_internal(obj, gb))$, and
$event(Node_B_internal(obj, gb)) ==> event(Node_A_start(obj))$.

These queries provide a content-based authentication relation for the object shared between the two parties.

To account for replay and session-mixing attacks, we also verify injective correspondence properties. The query $inj - event(Node_A_end(k, obj, ga, gb)) ==> inj - event(Node_B_end(k, obj, ga, gb))$ holds, meaning that each successful completion by A corresponds to a distinct matching completion by B for the same session key and object. Moreover, the stronger combined injective query also holds:
$inj - event(Node_A_end(k, obj, ga, gb)) ==> (inj - event(Node_B_end(k, obj, ga, gb))\&\&inj - event(Node_A_internal(obj, k, ga, gb))\&\&inj - event(Node_B_internal(obj, gb))\&\&inj - event(Node_A_start(obj)))$.

In this way, the protocol ensures session consistency and prevents multiple successful executions from being assigned to the same protocol run.

Finally, secrecy of the protected content is evaluated using the $query\ attacker(secr)$. The result not attacker(secr[]) is true shows that the adversary cannot derive the secret under the Dolev–Yao attacker model. Therefore, within the perfect cryptography abstraction used by ProVerif, the model satisfies the specified secrecy and content-based authentication requirements. These results support resistance against replay and MITM attacks with respect to the modeled session key, object binding, and event correspondences. We complement this formal analysis with the other security requirements and give an informal discussion of typical attacks against IPFS-based systems.

Privacy: The paper [10] focused on protecting the privacy of users retrieving content, while MITM-related attack scenarios were only partially addressed. In our proposed protocol, we have addressed this issue, however, an attacker could still construct dictionary in which $CID^1 - CID^3$ pairs are stored. The decoy version serves this purpose by sending fake messages to the network. Thus, even if an attacker profiles a user, they cannot determine which requests were real and which were decoys.

Forward secrecy: In the case of forward secrecy, long-lived keys are compromised. In this scenario, an attacker possessing the private key corresponding to PKB may decrypt the third protocol step. However, the session key itself is still derived through the Diffie–Hellman exchange and cannot be computed from the exchanged public values due to the Computational Diffie–Hellman (CDH) problem.

V. Implementation and Evaluation

To validate the proposed extensions, we developed a modular prototype in Node.js. The asymmetric encryption operations described in Section III are realized using ECIES (Elliptic Curve Integrated Encryption Scheme), while digital signatures use ECDSA (Elliptic Curve Digital Signature Algorithm) on the same curve. The implementation focuses on the three core contributions: the enhanced triple-hash protocol, the decoy request mechanism, and encrypted caching.

System Architecture and Cryptography

The prototype extends a custom P2P stack where the Bitswap engine was modified with nine new message types: `PRIVACY_CHALLENGE`, `PRIVACY_RESPONSE`, and `PRIVACY_BLOCK` for the authenticated handshake; `DECOY_REQUEST` and `DECOY_BLOCK` for traffic obfuscation; and `CACHE_REQUEST`, `CACHE_RESPONSE`, `CACHE_CHALLENGE`, and `CACHE_BLOCK` for the encrypted caching protocol. The base layer implements a Kademlia DHT for peer discovery and provider routing.

All cryptographic operations leverage the Node.js `crypto` library. Table I summarizes the selected primitives.

TABLE I
Cryptographic primitives and parameters.

Operation	Primitive / Parameters
Content hashing	SHA-256 (32-byte digest)
Identity keys	ECDSA on NIST P-256
Digital signatures	ECDSA-SHA256 (DER-encoded)
Asymmetric encryption	ECIES with HKDF-SHA256 key derivation and AES-256-GCM
Session encryption	AES-256-GCM (12-byte IV, 16-byte authentication tag)
Timing-safe comparison	`timingSafeEqual()`

Protocol Realization

1) Privacy Handshake and Decoys

The four-step handshake is integrated into the block exchange logic. MITM resistance is achieved through two complementary mechanisms: the owner's ECDSA signature over CID^2 proves possession without revealing CID^1, while ECIES encryption of the requester's response ensures that only the authenticated owner can recover CID^1 and the session key K. The decoy mechanism is interleaved with genuine traffic, for every successful retrieval, the node dispatches 1–3 decoy requests to random providers identified via the DHT. To an external observer, the ECIES-encrypted decoy responses are indistinguishable from legitimate data blocks.

2) Encrypted Caching

The caching layer implements a nested encryption scheme. The inner layer uses $H(\mathrm{CID}^1)$ as a symmetric key, ensuring that the cache node cannot decrypt the content without the requester's cooperation. The outer layer is bound to a session-specific key, providing forward secrecy for the cache-to-requester channel.

Experimental Results

The system was evaluated in a containerized multi-node environment (Owner, Cache, and Requester nodes). We measured the execution time for key operations over 50 iterations to quantify the computational overhead of the added privacy layers.

TABLE II
PROTOCOL EXECUTION TIMES (50 RUNS, LOCALHOST, 3 NODES).

Operation	Avg	Min	Max
TCP handshake (peer identification)	1.4 ms	1.0 ms	2.5 ms
File add + triple hash + DHT publish	4.4 ms	3.0 ms	7.2 ms
Privacy retrieval (4-step)	2.0 ms	1.5 ms	6.9 ms
Decoy request (4-step, discarded)	1.2 ms	0.7 ms	2.7 ms
Cache population (from owner)	2.9 ms	1.9 ms	4.8 ms
Cache retrieval (from cache node)	1.6 ms	1.3 ms	2.7 ms

The results (Table II) indicate that the privacy extensions introduce minimal latency. The complete 4-step handshake adds under 3 ms of protocol overhead on average, dominated by the ECIES operation and signature verification. Notably, decoy requests are processed faster than real requests as they bypass block store read and write operations, allowing the system to maintain a high-frequency noise floor with negligible resource impact. The source code of the prototype is publicly available [15].

VI. CONCLUSION

IPFS offers a promising foundation for distributed storage, yet its reliance on content-addressed lookups exposes users to privacy threats, as any node along the retrieval path can observe requested content identifiers. While the triple hashing scheme mitigates this exposure by decoupling lookup keys from content addresses, we have demonstrated that it remains vulnerable to MITM attacks without robust participant authentication.

In this paper, we presented three extensions to address these vulnerabilities. First, we enhanced the triple-hash exchange with signatures and asymmetric encryption, ensuring mutual authentication and protecting CID^1 even over compromised channels. Second, we introduced decoy requests that are cryptographically indistinguishable from genuine retrievals, significantly increasing the complexity of traffic-analysis-based profiling. Third, we designed an encrypted caching protocol that allows resource-rich nodes to serve content without gaining access to the underlying plaintext. Formal verification using ProVerif confirmed that our protocol achieves mutual authentication, key secrecy, and resistance to replay attacks under the Dolev-Yao adversarial model.

We validated our design through a functional prototype and experimental measurements in a containerized multi-node environment. The results demonstrate that the privacy extensions introduce negligible overhead. Furthermore, the decoy and caching mechanisms maintain performance levels suitable for real-world deployment. These findings suggest that strong privacy guarantees can be integrated into content-addressed networks without compromising efficiency.

Future work includes evaluating the protocol in larger network topologies with increased peer churn and adversarial presence, refining the adaptive decoy scheduling strategy based on traffic entropy analysis, and extending the formal model to provide a compositional security proof covering the caching sub-protocol jointly with the retrieval protocol.

REFERENCES

[1] D. A. B. Fernandes, L. F. B. Soares, J. V. Gomes, M. M. Freire, and P. R. M. Inácio, "Security issues in cloud environments: A survey," *Int. J. Inf. Security*, vol. 13, no. 2, pp. 113–170, 2014.

[2] I. Bashir, "Mastering Blockchain: A deep dive into distributed ledgers, consensus protocols, smart contracts, and the future of the blockchain", *3rd ed. Birmingham, UK: Packt Publishing*, 2020.

[3] J. Benet, "IPFS - Content addressed, versioned, P2P file system," *arXiv preprint arXiv:1407.3561*, 2014.

[4] E. Daniel and F. Tschorsch, "IPFS and friends: A qualitative comparison of next generation peer-to-peer data networks," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 1, pp. 31–52, 2022.

[5] D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, B. Gipp, and Y. Psaras, "Design and evaluation of IPFS: A storage layer for the decentralized web," in *Proc. ACM SIGCOMM*, 2022, pp. 739–752.

[6] S. Henningsen, M. Florian, S. Rust, and B. Scheuermann, "Mapping the interplanetary filesystem," in *Proc. IFIP Networking Conf.*, 2020, pp. 289–297.

[7] L. Balduf, S. Henningsen, M. Florian, S. Rust, and B. Scheuermann, "Monitoring data requests in decentralized data storage systems: A case study of IPFS," in *Proc. IEEE 42nd Int. Conf. Distributed Computing Systems (ICDCS)*, 2022, pp. 658–668.

[8] T. Cholez and C.-L. Ignat, "Sybil attack strikes again: Denying content access in IPFS with a single computer," in *Proc. 19th Int. Conf. Availability, Reliability and Security (ARES)*, ACM, 2024.

[9] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *Proc. 13th USENIX Security Symp.*, 2004, pp. 303–320.

[10] T. Katsantas, Y. Thomas, C. Karapapas, and G. Xylomenos, "Enhancing IPFS privacy through triple hashing," in *Proc. IEEE Symp. Computers and Communications (ISCC)*, IEEE, 2024, pp. 1–6.

[11] A. De la Rocha, D. Dias, and Y. Psaras, "Accelerating content routing with bitswap: A multi-path file transfer protocol in IPFS and Filecoin," *arXiv preprint arXiv:2111.04637*, 2021.

[12] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in *Peer-to-Peer Systems (IPTPS)*, Berlin, Heidelberg: Springer, 2002, pp. 53–65.

[13] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, 2016.

[14] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, 1983.

[15] C. Nagy, "Secure Data Exchange Protocol for IPFS – Prototype Implementation," GitHub repository, 2026. [Online]. Available: https://github.com/ncsn/Secure_Data_Exchange_Protocol_for_IPFS.